

DBMS LAB – TASK 9 – CSE 3

INTRODUCTION TO PL/SQL / STORED PROGRAMS

Using **TaxationDB**

1. Aim

To understand and implement stored-program concepts using the Taxation Database, including variables, conditional statements, loops, procedures, functions, cursors and error handling in Oracle PL/SQL and MySQL Stored Programs.

2. Objectives

After completing this laboratory, students should be able to:

- Understand why programming logic is required inside a database.
 - Understand the basic structure of a PL/SQL block.
 - Declare and use variables.
 - Retrieve database values into variables.
 - Use **IF**, **IF-ELSE** and **CASE**.
 - Implement loops.
 - Create and execute stored procedures.
 - Pass parameters to procedures.
 - Create and execute stored functions.
 - Understand cursors.
 - Handle runtime errors using exceptions/handlers.
 - Implement the same database logic in Oracle and MySQL.
-

3. Prerequisites

Students should know:

- SELECT
 - INSERT
 - UPDATE
 - DELETE
 - WHERE
 - Aggregate functions
 - GROUP BY
 - Subqueries
 - Views
 - Primary key and foreign key
 - Basic SQL
 - Basic understanding of transactions
-

4. How to Start

4.1 Oracle PL/SQL

Use:

Oracle Database + Oracle SQL Developer

Connect to the Oracle schema/user containing the TaxationDB tables.

First verify:

```
SELECT * FROM Taxpayer;
```

```
SELECT * FROM Income_Category;
```

```
SELECT * FROM Financial_Year;
```

```
SELECT * FROM Income_Record;
```

Enable output

For programs using `DBMS_OUTPUT.PUT_LINE`:

```
SET SERVEROUTPUT ON;
```

In SQL Developer, students may also need to enable the DBMS Output panel for the connection.

4.2 Your First Oracle PL/SQL Program

Start with the smallest possible PL/SQL block:

```
BEGIN

    DBMS_OUTPUT.PUT_LINE('Welcome to TaxationDB');

END;

/
```

The structure is:

```
DECLARE

    declarations

BEGIN

    executable statements

EXCEPTION

    error handling

END;

/
```

`DECLARE` and `EXCEPTION` are optional.

Therefore, the simplest block is:

```
BEGIN
```

```
-- executable statements  
END;  
/  


---


```

4.3 MySQL Stored Programs

Use:

MySQL Server + MySQL Workbench

Select the database:

```
USE taxation_db;
```

Verify:

```
SHOW TABLES;
```

Unlike Oracle, MySQL does not use an anonymous PL/SQL block like:

```
BEGIN
```

```
...
```

```
END;
```

```
/
```

Instead, programming logic is normally placed inside a stored procedure, stored function, trigger, etc.

Start with:

```
DELIMITER //
```

```
CREATE PROCEDURE hello_taxation()
```

```
BEGIN
```

```
    SELECT 'Welcome to TaxationDB';
```

END //

DELIMITER ;

Execute it:

CALL hello_taxation();

Important

In MySQL:

CALL procedure_name();

is used to execute a procedure.

In Oracle:

EXEC procedure_name;

or

BEGIN

 procedure_name;

END;

/

can be used to execute a procedure.

5. Concepts – Oracle PL/SQL and MySQL

5.1 Variables

Oracle

DECLARE

 v_income NUMBER;

```
BEGIN
```

```
    v_income := 750000;
```

```
    DBMS_OUTPUT.PUT_LINE(v_income);
```

```
END;
```

```
/
```

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE show_income()
```

```
BEGIN
```

```
    DECLARE v_income DECIMAL(12,2);
```

```
    SET v_income = 750000;
```

```
    SELECT v_income;
```

```
END //
```

```
DELIMITER ;
```

Execute:

```
CALL show_income();
```

5.2 Reading a Value from a Table

Suppose we want the annual income of taxpayer 1.

Oracle

DECLARE

 v_income Taxpayer.annual_income%TYPE;

BEGIN

 SELECT annual_income

 INTO v_income

 FROM Taxpayer

 WHERE taxpayer_id = 1;

 DBMS_OUTPUT.PUT_LINE('Annual Income: ' || v_income);

END;

/

MySQL

DELIMITER //

CREATE PROCEDURE get_taxpayer_income(IN p_taxpayer_id INT)

BEGIN

 DECLARE v_income DECIMAL(12,2);

 SELECT annual_income

 INTO v_income

```
FROM Taxpayer

WHERE taxpayer_id = p_taxpayer_id;

SELECT v_income AS annual_income;

END //

DELIMITER ;

Execute:

CALL get_taxpayer_income(1);
```

5.3 IF–ELSE

Suppose the taxation system classifies income into different categories.

Oracle

```
DECLARE

    v_income NUMBER := 850000;

BEGIN

    IF v_income > 1000000 THEN

        DBMS_OUTPUT.PUT_LINE('High Income');

    ELSIF v_income >= 500000 THEN

        DBMS_OUTPUT.PUT_LINE('Medium Income');
```


ELSE

DBMS_OUTPUT.PUT_LINE('Low Income');

END IF;

END;

/

MySQL

DELIMITER //

CREATE PROCEDURE classify_income(IN p_income DECIMAL(12,2))

BEGIN

IF p_income > 1000000 THEN

SELECT 'High Income' AS category;

ELSEIF p_income >= 500000 THEN

SELECT 'Medium Income' AS category;

ELSE

SELECT 'Low Income' AS category;

END IF;

END //

DELIMITER ;

Execute:

CALL classify_income(850000);

5.4 CASE Statement

Oracle

DECLARE

 v_income NUMBER := 850000;

BEGIN

 CASE

 WHEN v_income > 1000000 THEN

 DBMS_OUTPUT.PUT_LINE('High Income');

 WHEN v_income >= 500000 THEN

 DBMS_OUTPUT.PUT_LINE('Medium Income');

 ELSE

 DBMS_OUTPUT.PUT_LINE('Low Income');

 END CASE;

END;

/

MySQL

DELIMITER //

CREATE PROCEDURE classify_income_case(

IN p_income DECIMAL(12,2)

)

BEGIN

CASE

WHEN p_income > 1000000 THEN

SELECT 'High Income' AS category;

WHEN p_income >= 500000 THEN

SELECT 'Medium Income' AS category;

ELSE

SELECT 'Low Income' AS category;

END CASE;

END //

DELIMITER ;

5.5 Loops

Loops allow database programs to execute a statement repeatedly.

Oracle – Basic LOOP

DECLARE

 v_count NUMBER := 1;

BEGIN

 LOOP

 DBMS_OUTPUT.PUT_LINE('Count = ' || v_count);

 v_count := v_count + 1;

 EXIT WHEN v_count > 5;

 END LOOP;

END;

/

MySQL

DELIMITER //

CREATE PROCEDURE count_numbers()

```
BEGIN

DECLARE v_count INT DEFAULT 1;

count_loop: LOOP

    SELECT v_count;

    SET v_count = v_count + 1;

    IF v_count > 5 THEN
        LEAVE count_loop;
    END IF;

END LOOP;

END //
```

```
DELIMITER ;
```

5.6 Stored Procedures

A procedure stores a reusable set of database operations.

Oracle

```
CREATE OR REPLACE PROCEDURE show_taxpayer(  
    p_taxpayer_id IN NUMBER  
)
```

IS

```
    v_name Taxpayer.full_name%TYPE;
```

```
BEGIN
```

```
    SELECT full_name
```

```
    INTO v_name
```

```
    FROM Taxpayer
```

```
    WHERE taxpayer_id = p_taxpayer_id;
```

```
    DBMS_OUTPUT.PUT_LINE('Taxpayer: ' || v_name);
```

```
END;
```

```
/
```

Execute:

```
EXEC show_taxpayer(1);
```

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE show_taxpayer(  
    IN p_taxpayer_id INT
```

```
)  
  
BEGIN  
  
    DECLARE v_name VARCHAR(100);  
  
    SELECT full_name  
    INTO v_name  
    FROM Taxpayer  
    WHERE taxpayer_id = p_taxpayer_id;  
  
    SELECT v_name AS taxpayer_name;  
  
END //
```

DELIMITER ;

Execute:

```
CALL show_taxpayer(1);
```

5.7 Procedure Parameters

There are three important parameter modes in Oracle:

IN

OUT

IN OUT

Oracle

```
CREATE OR REPLACE PROCEDURE get_income(
```

```
    p_taxpayer_id IN NUMBER,
```

```
    p_income OUT NUMBER
```

```
)
```

```
IS
```

```
BEGIN
```

```
    SELECT annual_income
```

```
    INTO p_income
```

```
    FROM Taxpayer
```

```
    WHERE taxpayer_id = p_taxpayer_id;
```

```
END;
```

```
/
```

MySQL

MySQL also supports:

IN

OUT

INOUT

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE get_income(
```



```
    IN p_taxpayer_id INT,  
    OUT p_income DECIMAL(12,2)  
)  
BEGIN  
  
    SELECT annual_income  
    INTO p_income  
    FROM Taxpayer  
    WHERE taxpayer_id = p_taxpayer_id;  
  
END //  
  
DELIMITER ;  
  
Execute:  
  
CALL get_income(1, @income);  
  
SELECT @income;
```

5.8 Stored Functions

A function returns a value.

Oracle

```
CREATE OR REPLACE FUNCTION get_annual_income(  
    p_taxpayer_id NUMBER
```

```
)  
RETURN NUMBER  
IS  
    v_income NUMBER;  
BEGIN  
  
    SELECT annual_income  
    INTO v_income  
    FROM Taxpayer  
    WHERE taxpayer_id = p_taxpayer_id;  
  
    RETURN v_income;  
  
END;  
/  

```

Use:

```
SELECT get_annual_income(1)  
FROM dual;
```

MySQL

```
DELIMITER //
```

```
CREATE FUNCTION get_annual_income(  
    p_taxpayer_id INT
```

```
)  
RETURNS DECIMAL(12,2)  
DETERMINISTIC  
BEGIN  
  
    DECLARE v_income DECIMAL(12,2);  
  
    SELECT annual_income  
    INTO v_income  
    FROM Taxpayer  
    WHERE taxpayer_id = p_taxpayer_id;  
  
    RETURN v_income;  
  
END //  
  
DELIMITER ;  
Use:  
SELECT get_annual_income(1);
```

5.9 Cursor – Basic Idea

A cursor is useful when a program needs to process multiple rows one at a time.

For example:

Display the names and annual incomes of all active taxpayers one by one.

Oracle

DECLARE

CURSOR taxpayer_cursor IS

SELECT full_name, annual_income

FROM Taxpayer

WHERE is_active = 1;

BEGIN

FOR taxpayer_rec IN taxpayer_cursor LOOP

DBMS_OUTPUT.PUT_LINE(

taxpayer_rec.full_name || ' - ' ||

taxpayer_rec.annual_income

);

END LOOP;

END;

/

MySQL

DELIMITER //

```
CREATE PROCEDURE display_taxpayers()
BEGIN

    DECLARE done INT DEFAULT FALSE;
    DECLARE v_name VARCHAR(100);
    DECLARE v_income DECIMAL(12,2);

    DECLARE taxpayer_cursor CURSOR FOR
        SELECT full_name, annual_income
        FROM Taxpayer
        WHERE is_active = 1;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET done = TRUE;

    OPEN taxpayer_cursor;

    read_loop: LOOP

        FETCH taxpayer_cursor
        INTO v_name, v_income;

        IF done THEN
```

```
        LEAVE read_loop;

    END IF;

    SELECT v_name, v_income;

END LOOP;

CLOSE taxpayer_cursor;

END //
```

DELIMITER ;

5.10 Exception / Error Handling

Oracle

```
DECLARE

    v_income NUMBER;

BEGIN

    SELECT annual_income

    INTO v_income

    FROM Taxpayer

    WHERE taxpayer_id = 9999;
```

```
DBMS_OUTPUT.PUT_LINE(v_income);
```

```
EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Taxpayer not found');
```

```
WHEN OTHERS THEN
```

```
    DBMS_OUTPUT.PUT_LINE('An error occurred');
```

```
END;
```

```
/
```

MySQL

MySQL uses handlers rather than Oracle's **EXCEPTION** section.

```
DELIMITER //
```

```
CREATE PROCEDURE find_taxpayer(
```

```
    IN p_taxpayer_id INT
```

```
)
```

```
BEGIN
```

```
    DECLARE v_name VARCHAR(100);
```

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
BEGIN

    SELECT 'Taxpayer not found' AS message;

END;

SELECT full_name
INTO v_name
FROM Taxpayer
WHERE taxpayer_id = p_taxpayer_id;

SELECT v_name AS taxpayer_name;

END //
```

DELIMITER ;

6. Laboratory Activities

Part A – Getting Started

Task 1 – Oracle

Create an anonymous PL/SQL block that displays:

Welcome to TaxationDB

using:

DBMS_OUTPUT.PUT_LINE

Task 2 – MySQL

Create a stored procedure named:

welcome_taxation

that displays:

Welcome to TaxationDB

Execute it using **CALL**.

Task 3 – Oracle

Declare variables for:

- taxpayer name
- annual income

Retrieve the values of a particular taxpayer using **SELECT INTO** and display them.

Task 4 – MySQL

Create a procedure that accepts **taxpayer_id** as an **IN** parameter and displays:

- taxpayer name
 - annual income
-

Task 5 – Both Versions

Implement the same taxpayer-income retrieval operation in:

1. Oracle PL/SQL
2. MySQL Stored Procedure

Compare the syntax and execution method.

7. Level 0 Questions

Question 1

Using **TaxationDB**, write an Oracle PL/SQL block and a MySQL stored procedure to calculate and display the 10% of the annual income of a specified taxpayer.

Question 2

Create a procedure that accepts a **taxpayer_id** and displays:

Taxpayer Name

PAN Number

Occupation

Annual Income

Implement it in both Oracle and MySQL.

Question 3

Write a PL/SQL block/procedure to determine whether a taxpayer is:

Active

Inactive

based on the **is_active** column.

Question 4

Create a stored function that receives an income amount and returns:

'Taxable'

if the amount is greater than 0, otherwise:

'Non-Taxable'

Implement the function in both Oracle and MySQL.

Question 5

Write a program that uses a loop to display the numbers from 1 to 10.

Implement it in:

- Oracle PL/SQL
 - MySQL Stored Procedure
-

8. Level 1 Questions

Question 1

Create a procedure named `classify_taxpayer_income` that accepts a `taxpayer_id`.

Retrieve the taxpayer's `annual_income` and classify the taxpayer as:

Income > 1,000,000 → High Income

500,000 – 1,000,000 → Medium Income

Below 500,000 → Low Income

Implement using:

- `IF / ELSIF / ELSE` in Oracle.
 - `IF / ELSEIF / ELSE` in MySQL.
-

Question 2

Create a function that accepts a `taxpayer_id` and returns the taxpayer's annual income.

Then write a query to display:

full_name

annual_income

for all taxpayers using the function.

Implement the solution in both Oracle and MySQL.

9. Level 2 Questions

Question 1 – Cursor

The taxation department wants to review all active taxpayers.

Create a cursor-based program that processes all active taxpayers and displays:

Taxpayer Name

Annual Income

Occupation

Implement using:

- Oracle cursor
 - MySQL cursor + **NOT FOUND** handler
-

Question 2 – Exception / Handler

Create a procedure that accepts a **taxpayer_id**.

If the taxpayer exists, display:

Taxpayer Name

Annual Income

If the taxpayer does not exist, display:

Taxpayer not found

Implement the solution using:

- Oracle `NO_DATA_FOUND`
 - MySQL `NOT FOUND` handler
-

10. Optional Questions

Optional 1 – Procedure with UPDATE

Create a procedure that accepts:

`taxpayer_id`

new annual income

and updates the taxpayer's annual income.

Before updating, verify that the taxpayer exists.

Optional 2 – CASE

Create a procedure that accepts a taxpayer's annual income and uses `CASE` to classify the income into:

Low

Medium

High

Optional 3 – Income Records

Create a procedure that accepts a `taxpayer_id` and displays all corresponding records from `Income_Record`.

Display:

Income Source

Category

Amount

Received Date

Financial Year

Optional 4 – Cursor Processing

Create a cursor that processes all records in `Income_Record`.

For every record, display:

Income ID

Income Source

Amount

Financial Year

Optional 5 – Function with Taxation Logic

Create a function that accepts an income amount and returns a classification according to the following laboratory-defined rules:

Income $\leq$ 500000 → No Tax Category

500001–1000000 → Moderate Income

Above 1000000 → High Income

Use the function in a `SELECT` statement to display the classification for every taxpayer.

11. Important Oracle vs MySQL Comparison

Students should record this comparison in their lab observation.

Concept	Oracle PL/SQL	MySQL Stored Programs
Basic programming block	<code>DECLARE → BEGIN → EXCEPTION → END</code>	Usually inside procedure/function
Output	<code>DBMS_OUTPUT.PUT_LINE()</code>	<code>SELECT</code>
Variable declaration	<code>v_income NUMBER;</code>	<code>DECLARE v_income DECIMAL(12,2);</code>
Assignment	<code>:=</code>	<code>SET</code>
Query into variable	<code>SELECT ... INTO</code>	<code>SELECT ... INTO</code>
Conditional	<code>IF / ELSIF / ELSE</code>	<code>IF / ELSEIF / ELSE</code>
Alternative	<code>CASE</code>	<code>CASE</code>
Procedure	<code>CREATE OR REPLACE PROCEDURE</code>	<code>CREATE PROCEDURE</code>
Execute procedure	<code>EXEC / PL/SQL block</code>	<code>CALL</code>

Function	CREATE OR REPLACE FUNCTION	CREATE FUNCTION
Return value	RETURN	RETURN
Cursor	CURSOR	CURSOR
Error handling	EXCEPTION	DECLARE ... HANDLER
No row found	NO_DATA_FOUND	NOT FOUND
Statement delimiter	/ commonly used to execute block	DELIMITER // commonly used

13. GitHub Submission

StudentID-repo

|

└─ Week-9-Lab

|

└─ week9_plsql_oracle.sql

|

└─ week9_stored_programs_mysql.sql

